

条款22 当使用Pimpl惯用法,请在实现文件中定义特殊成员函数

① Pimpl的使用 (Pointer to implementation)

核心作用: 将头文件所有私有成员搬入cpp中,只留一个指针

目地: 加快编译、隐藏实现、稳定接口、解耦合

a. 未使用Pimpl时

Widget.h

```
#include <Gidget.h>
class Widget{
public: Widget();
private: Gidget g1;}
```

Gidget.h

```
class Gidget{
public: int data;}
```

Widget.cpp

```
#include <Widget>
Widget::Widget() {...};
```

main.cpp

```
#include <Widget.h>
int main(){ Widget w;}
```

1° 性能有问题: #include本质只是展开头文件

只要改动.h, .cpp文件就需要重新编译

2° 实现暴露: .h文件声明password对象,直接导致安全问题

b. 使用 Pimpl 时

Widget.h

```
class Widget{  
    public: Widget();  
           ~Widget();  
    private: struct Impl;  
            Impl* pImpl();  
};
```

main.cpp

```
#include <Widget.h>  
int main() { Widget w; }
```

Gidget.h

```
class Gidget{  
    public: int data;}
```

Widget.cpp

```
#include <Gidget.h>  
#include <Widget.h>  
struct Widget::Impl { Gidget  
                      g; };  
Widget::Widget(): pImpl(  
    new Impl) {}  
Widget::~~Widget() { delete  
    pImpl; }
```

1° 头文件不再需要大量 #include (vector, string 等等)
只重新编译 .cpp

2° 隐藏实现细节, 保护私有成员

3° 稳定 ABI (二进制兼容)

② 使用 unique_ptr 改写 pImpl

如果 Widget 的析构/移动构造/移动赋值在 Impl 还未定义处就定义了, 会报错

原因: `unique_ptr`析构时 默认删除器会使用`delete`来销毁
内置于`unique_ptr`的原始指针 然后在`delete`之前 通常会使用
C++11特性`static_assert`来确保原始指针指向的类型不是一
个未完成类型

移动操作同理: `pImpl = std::move(rhs.pImpl)`
也要先析构的对象

③使用`shared_ptr` 改写`pImpl`

`unique_ptr`存在的问题. `shared_ptr`都不存在

原因在于两者支持自定义删除器的方式不同